

Browser History Navigation Simulator

1. Introduction

The Browser History Navigation Simulator is a computer program designed to replicate the "Back" and "Forward" button logic found in modern web browsers. The project leverages the Stack data structure, which is ideal for this application because it follows the Last-In, First-Out (LIFO) principle. By using two separate stacks, the program can efficiently track where a user has been and where they can return to.

2. Theoretical Background

A Stack is a linear data structure that serves as a collection of elements with two principal operations:

- **Push:** Adds an element to the collection.
- **Pop:** Removes the most recently added element that was not yet removed.

In browser navigation:

- The **Back Stack** stores the history of pages visited.
- The **Forward Stack** stores pages that were moved to the "future" when a user clicked the "Back" button.

3. System Design and Logic

3.1 Data Structures

The project uses a struct to define the Stack:

- `char pages[PAGES][URL]:` A two-dimensional array of characters. This effectively acts as an array of strings, where each string is a URL.
- `int top:` An integer that tracks the index of the top-most element in the stack. A value of -1 indicates an empty stack.

3.2 Operation Logic

The core functionality is divided into three main navigation states:

A. Visiting a New Page

When a user types a new URL:

1. The current page is pushed onto the `back_stack`.
2. The `front_stack` is cleared (resetting `top` to -1). This is crucial because visiting a new site creates a new timeline, making previous "Forward" history invalid.
3. The `current_url` variable is updated with the new input.

B. Moving Backward

When the user selects "Back":

1. The `current_url` is pushed onto the `front_stack`.
2. The top element is popped from the `back_stack` and becomes the new `current_url`.

C. Moving Forward

When the user selects "Forward":

1. The `current_url` is pushed onto the `back_stack`.
2. The top element is popped from the `front_stack` and becomes the new `current_url`.

4. Algorithm and Pseudo Code

4.1 Push Algorithm

1. Start
2. If `top` equals `MAX_SIZE - 1`, print "Stack Overflow" and return.
3. Increment `top` by 1.
4. Copy the URL string into `pages[top]`.
5. End

4.2 Pop Algorithm

1. Start
2. If `top` equals -1, print "Stack Underflow" and return.
3. Copy the string at `pages[top]` to the destination variable.
4. Decrement `top` by 1.
5. End

4.3 Pseudo Code for Main Loop

```
INITIALIZE back_stack and front_stack with top = -1

SET current_url = "HOME"

REPEAT

    DISPLAY menu options and current_url

    INPUT user_choice

    SWITCH user_choice:

        CASE 1 (New Site):

            PUSH current_url to back_stack

            RESET front_stack (top = -1)

            INPUT new URL into current_url

        CASE 2 (Back):

            IF back_stack is empty:

                PRINT "No back history"

            ELSE:

                PUSH current_url to front_stack

                POP from back_stack into current_url

        CASE 3 (Forward):

            IF front_stack is empty:

                PRINT "No forward history"

            ELSE:

                PUSH current_url to back_stack

                POP from front_stack into current_url

        CASE 4: EXIT

    UNTIL exit signal received
```

5. Functional Requirements

- **Navigation:** Must allow moving between at least 100 pages.
- **Memory Management:** Use of `strcpy` to handle string transfers between the current state and the stacks.
- **Input Validation:** The program checks if the user is at the "start" or "end" of their history to prevent invalid pops.

6. Conclusion

This project successfully demonstrates the utility of the Stack data structure in a real-world software engineering context. By managing two distinct stacks, the program maintains a seamless history of user activity, ensuring that navigation is logical, predictable, and memory-efficient.

CODE-

```
#include <stdio.h>
#include <string.h>
#define PAGES 100 // max number of pages
#define URL 50 // max length of url

// struct for stacks//
struct Stack
{
    char pages[PAGES][URL];
    int top;
};

//push fxn
void push (struct Stack *stack , char current_url[]){
    if(stack->top==PAGES-1){
        printf("Not enough memory.\n");
    }
    else{
        stack->top++;
        // copies current url into stack
        strcpy(stack->pages[stack->top], current_url);
    }
}

// pop fxn
void pop(struct Stack *stack , char destination[]){
    if(stack->top==0){
        printf("Underflow Condition");
    }
    else{
        // copies value of stack to destination
        strcpy(destination,stack->pages[stack->top]);
        stack->top--;
    }
}

int main(){
    printf("===== \n");
    printf("    BROWSER SIMULATOR    \n");
    printf("===== \n");
    // used variables
    int run_again=1; // for do while loop
    char current_url[URL]="HOME"; // starting page
    int cmd; // for switch case commands

    // front and back stack
    struct Stack front_stack;
    front_stack.top=-1;

    struct Stack back_stack;
    back_stack.top=-1;

    // in a loop use switch case to get user command 1- new site ,
    //2-back,3-forward,4-exit
    do{
        printf("-----MENU-----\n");
        printf("Enter 1 to visit new site.\n");
        printf("Enter 2 to go backward.\n");
        printf("Enter 3 to go forward.\n");
        printf("Enter 4 to exit.\n");
        printf("-----\n");
        printf("command : ");
        scanf("%d",&cmd);
        switch(cmd){
            case 1:
                /* case for new
                push current url in back
                clear front stack
                take input for new url*/

                push (&back_stack,current_url); //pushes current url in back stack
                front_stack.top=-1; // clears the front stack as when we search new page all pages in forward stack
                clean up
                printf("Enter url of site : ");
                scanf("%s",current_url); // take input for new url
                break;

            case 2:
                /* case for back
                push current url to front
                url=pop(back stack)*/

                if(back_stack.top==0){
                    printf("\n* * * * * \n ");
                    printf("You are already on first site.");
                    printf("\n* * * * * \n ");
                } // this condition means we are already at 1st page.
                else{
                    push(&front_stack,current_url); // we push current url into front stack
                    pop(& back_stack,current_url); // then pop and store value from back stack into current url
                }
                break;

            case 3:
                /*case for forward
                push current url to back
                url=pop(front)*/

                if(front_stack.top==0){
                    printf("\n* * * * * \n ");
                    printf("You are already on the last site.");
                    printf("\n* * * * * \n ");
                } // this condition means we are already at last page
                else{
                    push(&back_stack,current_url); //pushing current url into back stack
                    pop(&front_stack,current_url); // popping and storing value from front stack into current url
                }
                break;

            case 4:
                run_again=0;
                printf("===== \n");
                printf("    THANK YOU    \n");
                printf("===== \n");
                break;

            default:
                printf("Invalid Input.\n");
                break;
        }
        if(run_again){
            printf("\n* * * * * \n ");
            printf("You are currently on: %s",current_url);
            printf("\n* * * * * \n ");
        }
    } while(run_again);

    return 0;
}
```



```

=====
BROWSER SIMULATOR
=====
-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 1
Enter url of site : A

* * * * *
You are currently on: A
* * * * *

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 1
Enter url of site : B

* * * * *
You are currently on: B
* * * * *

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 1
Enter url of site : C

* * * * *
You are currently on: C
* * * * *

```

Inputing some urls:

first we enter -A

then, B

then, C

```

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 2

*****
You are currently on: B
*****

```

Using Back option

as you can see the last value we inputted was C when we go back from C the code shows B i.e the input prior to C.

```

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 1
Enter url of site : D

*****
You are currently on: D
*****

```

Entering new url after going back

here we go to new site named D after going back (visiting B).

so now its like- A→B→D

this clears the front stack.

```

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 2

*****
You are currently on: B
*****

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 2

*****
You are currently on: A
*****

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 2

*****
You are currently on: HOME
*****

```

Going backwards till home

as we go backwards we move from D to B to A and then finally home.

```
-----MENU-----
```

```
Enter 1 to visit new site.
```

```
Enter 2 to go backward.
```

```
Enter 3 to go forward.
```

```
Enter 4 to exit.
```

```
-----
```

```
command : 2
```

```
* * * * *
```

```
You are already on first site.
```

```
* * * * *
```

```
* * * * *
```

```
You are currently on: HOME
```

```
* * * * *
```

EDGE CASE

what if we try to go back even after home page

as you can see in this condition it told us we are already at first site.

Going Forward Till last page

as you can see we moved like,

A→B→D

```
-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 3

*****
You are currently on: A
*****

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 3

*****
You are currently on: B
*****

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 3

*****
You are currently on: D
*****
```

```

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 3

* * * * *
You are already on the last site.
* * * * *

* * * * *
You are currently on: D
* * * * *

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 3

* * * * *
You are already on the last site.
* * * * *

* * * * *
You are currently on: D
* * * * *

-----MENU-----
Enter 1 to visit new site.
Enter 2 to go backward.
Enter 3 to go forward.
Enter 4 to exit.
-----
command : 4
=====
THANK YOU
=====

```

EDGE CASE

what will happen if we try to go forward even after last site,
as the results shows code tells us that we are already at last
site so we cant go forward.